UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
DARCY RIBEIRO

JUAN AUGUSTO CRISTO HYGINO

Governance Gateway: Governança, Auditoria e Observabilidade em Tempo de Execução no Ecossistema MCP

Campos dos Goytacazes
2026

JUAN AUGUSTO CRISTO HYGINO

Governance Gateway: Governança, Auditoria e Observabilidade em Tempo de Execução no Ecossistema MCP

Monografia submetida como requisito parcial para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Profa. Dra. Annabel D.R. Tamariz

Campos dos Goytacazes
2026

Índice

1 Introdução **1**
1.1 Motivação . . . 2
1.2 Formulação do Problema . . . 2
1.2.1 Problema Geral . . . 2
1.2.2 Subproblemas . . . 3
1.3 Hipótese . . . 3
1.3.1 Hipótese Geral . . . 3
1.3.2 Hipóteses Específicas . . . 4
1.4 Objetivos da Pesquisa . . . 4
1.4.1 Objetivo Geral . . . 4
1.4.2 Objetivos Específicos . . . 4
1.5 Justificativa . . . 5
1.6 Método . . . 6
1.7 Organização . . . 6

2 Fundamentação Teórica **8**
2.1 Agentes Autônomos e a Arquitetura do Model Context Protocol (MCP) . . . 8
2.1.1 Agentes Baseados em Modelos de Linguagem e Invocação de Ferramentas 9
2.1.2 Topologia *Host-Client-Server* e Primitivas Operacionais do MCP . . . 10
2.1.3 Camada de Dados JSON-RPC 2.0, Ciclo de Sessão e Canais de Transporte 11
2.2 Superfície de Ataque e Modelagem de Ameaças em Ecossistemas MCP . . . 11
2.2.1 O Paradigma de Confiança Implícita e o Ciclo de Vida do Servidor . . . 12
2.2.2 Taxonomia de Vetores de Ataque: Camadas Cognitiva, Protocolar e de Infraestrutura . . . 12
2.2.3 A Lacuna Semântica e a Deriva de Privilégios . . . 14
2.3 Governança em Tempo de Execução e Controle de Acesso Baseado em Papéis 14
2.3.1 Princípio do Menor Privilégio e Monitores de Referência . . . 15
2.3.2 Modelo Formal de Controle de RBAC e Taxonomias de Permissão . . . 15
2.3.3 Máquina de Estados de Sessão e Elevação Supervisionada . . . 16
2.4 Auditoria Forense Imutável e Observabilidade Distribuída em Tempo Real . . . 16
2.4.1 Registro Estruturado e Encadeamento Criptográfico de Resumos . . . 17

2.4.2 Garantias de Integridade, Rastreabilidade Causal e Não-Repúdio 17
2.4.3 Telemetria Padronizada com OpenTelemetry e Rastreamento Distribuído 18
2.5 Trabalhos Relacionados 18
2.5.1 Estudos Empíricos de Medição, Qualidade e Segurança do Ecossistema MCP 18
2.5.2 Modelagem de Ameaças, Ciclo de Vida e Frameworks Hierárquicos de Defesa 19
2.5.3 Arquiteturas de Isolamento, Controle de Acesso em Tempo de Execução e Práticas Formais de RBAC 19
2.5.4 Discussão Comparativa e Síntese dos Trabalhos Relacionados 20

3 Modelo Conceitual do Artefato **22**
3.1 Módulo de Interceptação e Roteamento de Sessão 24
3.1.1 Adaptação dos Canais de Transporte 25
3.1.2 Decodificação e Extração de Contexto JSON-RPC 25
3.1.3 Filtragem de Catálogo e Imposição de Política (PEP) 25
3.2 Módulo de Governança Determinística, RBAC e Máquina de Estados (PDP) 26
3.2.1 Classificação de Ferramentas e Matriz RBAC 27
3.2.2 Máquina de Estados de Sessão (Níveis L0 a L3) 28
3.2.3 Elevação Supervisionada com Janela Temporal (*HITL*) 29
3.3 Módulo de Auditoria Forense Imutável e Verificação Criptográfica 29
3.3.1 Serialização Canônica e Registro Estruturado (JSONL) 30
3.3.2 Encadeamento Criptográfico Recursivo (SHA-256) 31
3.3.3 Integridade e Reconstrução Causal 31
3.4 Módulo de Observabilidade Distribuída e Telemetria em *Runtime* 32
3.4.1 Propagação de Contexto W3C e Rastreamento Distribuído 33
3.4.2 Métricas de Segurança e Desempenho 34
3.4.3 Correlação Forense-Telemetria e Exportação OTLP 34
3.5 Integração Conceitual do Modelo e Fundamentação nos Artigos Inspiradores . 34
3.5.1 Fluxo Operacional Integrado Fim a Fim (Ciclo de Vida da Requisição) . 35
3.5.2 Mapeamento Conceitual com os Artigos Inspiradores 35
3.5.3 Viabilidade de Engenharia Concorrente em Máquinas Virtuais 37

Referências Bibliográficas **38**

Capítulo 1

Introdução

A engenharia de software passa atualmente por uma transformação significativa com a evolução dos Modelos de Linguagem de Larga Escala (*Large Language Models* - LLMs) de interfaces conversacionais passivas para agentes autônomos orientados a objetivos. Diferente do modelo tradicional de consultas isoladas, esses agentes operam em ciclos contínuos de raciocínio, planejamento e ação, dividindo metas complexas em etapas menores e acionando rotinas externas em ambientes distribuídos (Bühler et al., 2026; Singh e Madisetti, 2026).

Para interagir com sistemas reais além de seus dados estáticos de treinamento, os agentes dependem da invocação de ferramentas (*tool calling*) e do acesso a fontes externas, como bancos de dados, sistemas de arquivos e interfaces de programação de aplicações (APIs) (Hou et al., 2025; Hasan et al., 2026). Inicialmente, essa integração sofria com forte fragmentação técnica, pois cada plataforma utilizava conectores proprietários e incompatíveis entre si. Para padronizar essa comunicação, a Anthropic lançou no final de 2024 o *Model Context Protocol* (MCP), um protocolo aberto cliente-servidor que rapidamente se consolidou como o padrão da indústria para conectar aplicações de IA (hospedeiros e clientes) a provedores de contexto e ferramentas (servidores) (Hou et al., 2025; Guo et al., 2025), conforme ilustrado na Figura 2.1.

Apesar de simplificar a interoperabilidade, a rápida adoção do MCP expôs uma superfície de ataque crítica (Guo et al., 2025; Hou et al., 2025). O protocolo opera sob uma postura de **confiança implícita**: após a conexão inicial, o agente obtém acesso irrestrito às ferramentas expostas pelo servidor, que executa operações com os mesmos privilégios do usuário no sistema operacional (Bühler et al., 2026; Singh e Madisetti, 2026). Estudos recentes demonstram que essa arquitetura está sujeita a vulnerabilidades de código, envenenamento semântico de ferramentas, ataques em cadeia de suprimentos e falhas de controle de acesso (Hasan et al., 2026; Jang e Lee, 2026; Zhang et al., 2025), cujas especificações técnicas e dados empíricos são detalhados no Capítulo 2.

Diante desse cenário, observa-se a ausência de um mecanismo unificado que combine a aplicação determinística do Princípio do Menor Privilégio em tempo de execução,

a geração de registros de auditoria forense confiáveis e a observabilidade padronizada do tráfego de mensagens. É nessa lacuna de pesquisa que este trabalho se desenvolve.

1.1 Motivação

O avanço recente dos modelos de linguagem impulsionou a adoção de agentes autônomos diretamente em ambientes corporativos e produtivos. Em setores como tecnologia, finanças e desenvolvimento de software, esses agentes já automatizam tarefas críticas, incluindo consultas e alterações em bancos de dados, gerenciamento de repositórios de código e interação com serviços em nuvem.

Nesse contexto, o protocolo MCP assumiu papel central por permitir que desenvolvedores conectem fontes de dados e ferramentas a modelos inteligentes com facilidade. Entretanto, priorizou-se a rapidez de integração em detrimento de mecanismos nativos de governança e segurança. Na prática, agentes corporativos frequentemente operam com permissões excessivas, podendo causar danos operacionais severos caso sofram manipulações externas ou desvios de execução.

Além dos riscos operacionais, a falta de controle rigoroso expõe as organizações a passivos regulatórios. Normas de proteção de dados (como a LGPD e a GDPR) e padrões de segurança da informação exigem que o acesso a recursos sensíveis seja restrito, registrado e auditável. Assim, a motivação deste trabalho reside na necessidade de viabilizar o uso corporativo seguro de agentes baseados em MCP, aliando controle de acesso estrito, rastreabilidade forense e monitoramento contínuo.

1.2 Formulação do Problema

A especificação oficial do protocolo MCP (Hou et al., 2025; Singh e Madisetti, 2026) padronizou o transporte e a troca de mensagens entre agentes e ferramentas, mas delegou as decisões de segurança aos aplicativos hospedeiros ou à confirmação manual do usuário. Essa abordagem gera fadiga de alertas em operações contínuas e mantém o ambiente exposto ao modelo de confiança implícita do protocolo, às limitações da lacuna semântica de isolamentos tradicionais (Bühler et al., 2026) e ao risco de delegar decisões de acesso ao próprio modelo de linguagem (Zhang et al., 2025). Diante desse cenário, a problemática desta investigação divide-se em um problema geral e quatro subproblemas diretamente correlacionados às hipóteses e aos objetivos do trabalho.

1.2.1 Problema Geral

O problema geral desta pesquisa consiste na ausência de uma camada intermediária de governança em tempo de execução para ecossistemas baseados no protocolo MCP que seja capaz de impor controle de acesso determinístico segundo o Princípio do Menor

Privilégio e o Controle de Acesso Baseado em Papéis (*Role-Based Access Control* - RBAC), assegurar auditoria forense imutável e fornecer observabilidade padronizada, sem exigir alterações no código-fonte dos clientes e servidores existentes e mantendo sobrecarga de latência imperceptível em produção.

1.2.2 Subproblemas

1. **Fragmentação no Mapeamento de Ameaças do Ecossistema:** Carência de uma sistematização prática que consolide as vulnerabilidades de código, de protocolo e de manipulação semântica (como o envenenamento de ferramentas e a deriva de privilégios) em cenários adversariais reprodutíveis para validação de defesas (Hasan et al., 2026; Hou et al., 2025; Jang e Lee, 2026);

2. **Falta de Mediação Transparente no Canal de Comunicação:** Inexistência de um mecanismo de interceptação não invasivo entre cliente e servidor que filtre chamadas maliciosas em tempo de execução sem demandar a refatoração de servidores comunitários ou legados (Singh e Madisetti, 2026; Guo et al., 2025);

3. **Ausência Integrada de Controle RBAC, Auditoria Imutável e Telemetria:** Exposição simultânea de ferramentas de leitura e de escrita na mesma sessão sem segregação formal de papéis (Zhang et al., 2025; Singh e Madisetti, 2026), somada à geração de registros locais voláteis vulneráveis à adulteração e à falta de observabilidade distribuída padronizada em tempo real (Jang e Lee, 2026);

4. **Compromisso entre Rigor de Segurança, Desempenho e Comportamento do Agente:** Dificuldade de bloquear ações não autorizadas de forma determinística mantendo latência inferior a um milissegundo e induzindo a adaptação segura do planejamento do agente (moldagem comportamental), sem degradar a fluidez operacional do sistema hospedeiro (Singh e Madisetti, 2026).

1.3 Hipótese

A hipótese científica adotada nesta pesquisa é estabelecida por meio de afirmações declarativas, dividindo-se em uma hipótese geral e quatro hipóteses específicas correspondentes.

1.3.1 Hipótese Geral

A introdução de uma camada intermediária de governança não invasiva no canal de comunicação do ecossistema MCP é capaz de mitigar de forma eficaz vetores de ataque adversariais, provendo controle de acesso, auditoria forense e telemetria sem comprometer a latência e a compatibilidade do sistema hospedeiro.

1.3.2 Hipóteses Específicas

1. A sistematização formal das vulnerabilidades do ecossistema MCP viabiliza a reprodução controlada de vetores de ataque representativos, como a elevação indevida de privilégios e o envenenamento de ferramentas;

2. O uso de uma arquitetura de interceptação transparente permite a mediação contínua das mensagens do protocolo sem exigir alterações no código-fonte de clientes e servidores legados;

3. A aplicação combinada de controle RBAC, trilhas de auditoria protegidas por encadeamento criptográfico e telemetria padronizada estabelece uma governança determinística e rastreável sobre as ações do agente;

4. A atuação dessa camada em tempo de execução bloqueia ações não autorizadas com sobrecarga de latência imperceptível (inferior a 1,0 ms por chamada), induzindo simultaneamente a moldagem comportamental no planejamento do agente.

1.4 Objetivos da Pesquisa

1.4.1 Objetivo Geral

O objetivo geral deste trabalho consiste em **projetar, implementar e validar experimentalmente uma camada de governança, auditoria forense e observabilidade em tempo de execução (*Governance Gateway*) para ecossistemas baseados no protocolo MCP**, mitigando as vulnerabilidades decorrentes da confiança implícita do protocolo por meio de políticas de menor privilégio (RBAC), registro criptograficamente íntegro de requisições e telemetria operacional em tempo real.

1.4.2 Objetivos Específicos

Para atingir o objetivo geral e validar as hipóteses formuladas, definem-se os seguintes objetivos específicos:

1. **Mapear e formalizar o modelo de ameaças do ecossistema MCP:** Consolidar os principais vetores de vulnerabilidade da literatura recente (Hasan et al., 2026; Hou et al., 2025; Jang e Lee, 2026; Bühler et al., 2026) em uma suíte controlada de testes adversariais;

2. **Projetar a arquitetura do gateway de governança:** Especificar e desenvolver uma estrutura de mediação transparente baseada em proxy capaz de interceptar o tráfego local e remoto do protocolo sem exigir alterações nos clientes ou servidores existentes;

3. **Implementar os três pilares funcionais de segurança:**

   - **Pilar I - Controle de Acesso Baseado em Papéis (RBAC):** Desenvolver um motor determinístico de regras que classifique ferramentas por perfil de acesso e efeito operacional, separando consultas de mutações e suportando elevação temporária sob supervisão humana (Singh e Madisetti, 2026; Zhang et al., 2025);
   - **Pilar II - Trilha de Auditoria Forense Imutável:** Implementar o registro estruturado de eventos com encadeamento criptográfico de resumos SHA-256 à prova de adulteração, assegurando integridade e não-repúdio;
   - **Pilar III - Observabilidade e Telemetria em Tempo Real:** Integrar a coleta e exportação de métricas e rastreamentos distribuídos no padrão Open-Telemetry para monitoramento contínuo em painel visual;

4. **Realizar avaliação experimental e medição de desempenho:** Submeter o protótipo a ensaios controlados para medir a taxa de bloqueio de ataques frente a sistemas desprotegidos, quantificar o acréscimo de latência e consumo de recursos, e verificar a ocorrência de moldagem comportamental nos agentes.

1.5 Justificativa

A realização desta pesquisa justifica-se por sua relevância técnica, acadêmica e prática. Do ponto de vista **técnico**, a mediação transparente no barramento do protocolo preserva total compatibilidade com clientes e servidores de terceiros, enquanto o uso de regras determinísticas de RBAC retira a decisão de segurança do modelo probabilístico (Zhang et al., 2025), preenchendo a lacuna semântica apontada na literatura (Bühler et al., 2026).

Sob a perspectiva **científica e acadêmica**, os estudos sobre o protocolo MCP são recentes (2025–2026) e concentram-se majoritariamente no diagnóstico de vulnerabilidades (Hasan et al., 2026; Guo et al., 2025; Hou et al., 2025). Este trabalho contribui ao integrar controle de acesso em tempo de execução, auditoria forense criptográfica e observabilidade distribuída em um artefato funcional validado experimentalmente.

No aspecto **prático**, a adoção corporativa de agentes autônomos exige proteção a dados críticos e conformidade com normas regulatórias (como LGPD e GDPR). A entrega de uma trilha de auditoria inviolável aliada à telemetria em tempo real oferece às organizações a confiabilidade necessária para operar agentes MCP em produção.

1.6 Método

Este trabalho adota como referencial metodológico a *Design Science Research* (DSR), voltada à construção e avaliação rigorosa de artefatos computacionais para solucionar problemas práticos relevantes. O método estrutura-se em cinco fases iterativas:

1. **Fase 1 - Identificação do Problema e Revisão Bibliográfica:** Levantamento do estado da arte sobre segurança, medição e controle de acesso no protocolo MCP (Hasan et al., 2026; Guo et al., 2025; Hou et al., 2025; Jang e Lee, 2026; Bühler et al., 2026; Singh e Madisetti, 2026; Zhang et al., 2025), resultando na formalização do modelo de ameaças;

2. **Fase 2 - Concepção Arquitetural e Modelagem Teórica:** Especificação do gateway de governança, definindo a máquina de estados de sessão, a matriz RBAC, a estrutura de encadeamento criptográfico de logs e o esquema de telemetria compatível com OpenTelemetry (Singh e Madisetti, 2026; Jang e Lee, 2026);

3. **Fase 3 - Engenharia e Implementação do Protótipo:** Construção do artefato funcional em linguagem de alta eficiência e execução em máquinas virtuais (JAVA/Elixir), integrando os módulos de interceptação RBAC, auditoria forense SHA-256 e exportação de telemetria;

4. **Fase 4 - Avaliação Experimental e Coleta de Métricas:** Execução de testes em bancada controlada para aferir a eficácia de segurança (taxa de bloqueio contra cenários adversariais frente ao cenário base desprotegido) e a sobrecarga de desempenho (latência adicionada e uso de CPU/memória);

5. **Fase 5 - Análise dos Resultados e Conclusão:** Tratamento quantitativo dos dados obtidos, comparação com as métricas da literatura de referência (Singh e Madisetti, 2026; Bühler et al., 2026) e consolidação das conclusões da monografia.

1.7 Organização

Esta monografia divide-se em seis capítulos:

- **Capítulo 1 - Introdução:** Contextualiza o tema e apresenta a motivação, a problemática declarativa, as hipóteses, os objetivos, a justificativa, a metodologia e a estrutura do trabalho;

- **Capítulo 2 - Fundamentação Teórica:** Detalha a especificação arquitetural do MCP, a modelagem de ameaças, os modelos de RBAC e menor privilégio, a auditoria forense criptográfica, a observabilidade com OpenTelemetry e a revisão crítica dos trabalhos relacionados;

- **Capítulo 3 - Modelo Conceitual do Artefato:** Apresenta o modelo conceitual do *Governance Gateway*, detalhando seus quatro módulos, fluxos de processos, vínculos de dependência e o mapeamento direto com os artigos inspiradores da literatura;
- **Capítulo 4 - Metodologia e Avaliação Experimental:** Apresenta o ambiente de testes, os cenários adversariais e o protocolo de medição;
- **Capítulo 5 - Resultados Experimentais e Discussão:** Analisa os resultados de eficácia de bloqueio, impacto de latência e moldagem comportamental;
- **Capítulo 6 - Considerações Finais:** Sintetiza as contribuições do estudo, suas limitações e as direções para trabalhos futuros.

Capítulo 2

Fundamentação Teórica

A viabilização de implementações mais seguras de agentes de inteligência artificial depende diretamente de um maior controle sobre o protocolo MCP, indo em contrapartida à filosofia de confiança implícita inicialmente assumida pelo protocolo. Além disso, a literatura recente expõe uma quantidade preocupante de vulnerabilidades em implementações reais do MCP, que abrangem desde falhas estruturais do próprio protocolo até comportamentos inseguros exclusivos de agentes autônomos.

Este capítulo apresenta o referencial teórico que fundamenta a engenharia de uma camada de governança, auditoria forense e observabilidade em tempo de execução para o *Model Context Protocol*, intitulada *Governance Gateway*. A narrativa deste capítulo articula os conceitos arquiteturais, criptográficos e de segurança de sistemas distribuídos utilizados neste trabalho com as pesquisas recentes que compõem a base bibliográfica da proposta.

A Seção 2.1 detalha o funcionamento de agentes autônomos e a especificação técnica do protocolo MCP sobre JSON-RPC 2.0. A Seção 2.2 examina a superfície de ataque decorrente do modelo de confiança implícita, sistematizando o ciclo de vida dos servidores, os vetores de exploração em três camadas e os conceitos de lacuna semântica e deriva de privilégios. A Seção 2.3 fundamenta a mediação determinística via Princípio do Menor Privilégio, Controle de Acesso Baseado em Papéis (RBAC) e máquinas de estados de sessão. A Seção 2.4 aborda a auditoria forense imutável com encadeamento SHA-256 e a observabilidade distribuída com OpenTelemetry. Por fim, a Seção 2.5 analisa os sete trabalhos relacionados da literatura e explicita os métodos que inspiram esta monografia.

2.1 Agentes Autônomos e a Arquitetura do Model Context Protocol (MCP)

A transição dos modelos de linguagem de simples interfaces conversacionais para agentes capazes de interagir com o mundo exterior transformou a arquitetura das aplicações modernas de inteligência artificial. Para que um modelo possa consultar bancos

de dados, manipular arquivos locais ou acionar serviços em nuvem de forma padronizada, tornou-se necessária a criação de um protocolo de comunicação universal que eliminasse a necessidade de integrações manuais e específicas para cada ferramenta. Compreender como esse protocolo estrutura a troca de mensagens e delega ações ao ambiente operacional é o primeiro passo para identificar onde os controles de segurança devem ser posicionados.

Para construir essa base arquitetural de forma progressiva, esta seção inicia na Subseção 2.1.1 apresentando a evolução dos agentes autônomos orientados à invocação de ferramentas e o surgimento do MCP como padrão aberto de interoperabilidade. Em seguida, a Subseção 2.1.2 detalha a divisão de responsabilidades na topologia distribuída entre hospedeiro, cliente e servidor, bem como as primitivas operacionais expostas ao modelo. Por fim, a Subseção 2.1.3 especifica a estrutura das mensagens na camada JSON-RPC 2.0, o ciclo de vida das sessões e o funcionamento dos canais de transporte local e remoto.

2.1.1 Agentes Baseados em Modelos de Linguagem e Invocação de Ferramentas

Os Modelos de Linguagem de Larga Escala (LLMs) evoluíram de geradores passivos de texto restritos ao conhecimento estático de seu treinamento para atuar como o núcleo decisório de agentes autônomos. Fundamentados no paradigma de raciocínio e ação (*Reasoning and Acting* - ReAct), esses agentes operam em um laço iterativo de três etapas: (i) **Percepção e Observação**, recebendo instruções do usuário e dados do ambiente; (ii) **Raciocínio e Planejamento**, decompondo objetivos complexos em subtarefas sequenciais; e (iii) **Ação**, interagindo com sistemas externos para consultar informações em tempo real ou produzir alterações de estado (Singh e Madisetti, 2026).

O mecanismo técnico que viabiliza essa transição para a execução prática é a invocação de ferramentas (*tool calling*). A aplicação no *host* injeta na janela de contexto do LLM declarações estruturadas em esquema JSON descrevendo o nome da função, sua finalidade semântica e a tipagem de seus argumentos. Quando uma tarefa exige ação externa, o modelo emite um objeto JSON indicando a função escolhida e seus parâmetros; o ambiente executa a rotina no sistema operacional ou em uma API remota e devolve o resultado ao modelo para continuidade do raciocínio.

Segundo Hou et al. (Hou et al., 2025) e Hasan et al. (Hasan et al., 2026), a integração entre modelos e ferramentas atravessou quatro fases históricas: (1) **Acoplamento Manual de APIs (Até 2022)**, baseado em conectores proprietários codificados ponto a ponto; (2) **Ecossistemas Fechados de Plugins (2023)**, restritos à plataforma de um único fornecedor de modelo; (3) **Frameworks de Orquestração (2023–2024)**, como *LangChain* e *AutoGen*, que geraram o efeito silo (problema $N \times M$, exigindo reimplemen-

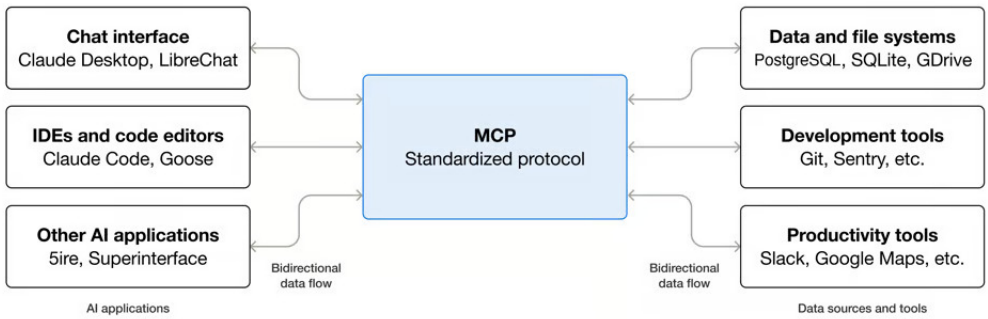
tações em cada plataforma); e (4) **Padronização Aberta via MCP (Final de 2024 em diante)**, lançada pela Anthropic e inspirada no *Language Server Protocol* (LSP), que reduziu a fragmentação $N \times M$ a um problema linear $N + M$.

2.1.2 Topologia *Host-Client-Server* e Primitivas Operacionais do MCP

Para desacoplar o raciocínio do modelo da implementação das ferramentas, a arquitetura do MCP define três entidades distribuídas (Hou et al., 2025; Guo et al., 2025):

- **MCP Host:** Aplicação voltada ao usuário onde o agente reside (ex.: *Claude Desktop*, *Cursor* ou *VS Code*), responsável por gerenciar o contexto do LLM, coordenar múltiplos clientes $(1 : N)$ e agregar catálogos;
- **MCP Client:** Conector interno instanciado pelo *host* que mantém uma sessão dedicada e isolada $(1 : 1)$ com um único servidor, gerenciando negociação de versão e roteamento bidirecional;
- **MCP Server:** Processo independente (local ou remoto) que encapsula sistemas de arquivos, bancos de dados ou APIs externas e expõe capacidades padronizadas.

Figura 2.1: Arquitetura de Integração do Model Context Protocol.



Fonte: Anthropic (2024)

O servidor disponibiliza três primitivas segundo a entidade controladora: (1) **Ferramentas (`tools`)**, funções executáveis invocadas pelo LLM capazes de produzir efeitos colaterais, definidas por nome, descrição documental (*docstring*) e esquema `inputSchema`; (2) **Recursos (`resources`)**, fontes de dados somente-leitura endereçadas por URIs; e (3) **Modelos de Instrução (`prompts`)**, templates parametrizados para fluxos recorrentes. Internamente, o servidor divide-se em cinco componentes: *Metadados*, *Configuração*, *Lista*

de Ferramentas, *Lista de Recursos* e *Templates de Prompt*, enquanto o cliente dispõe das primitivas de delimitação de diretórios (`roots`), amostragem (`sampling`) e elicitação (`elicitation`) (Hou et al., 2025; Hasan et al., 2026).

2.1.3 Camada de Dados JSON-RPC 2.0, Ciclo de Sessão e Canais de Transporte

O MCP separa a camada de dados da camada de transporte. A camada de dados adota o protocolo **JSON-RPC 2.0** em UTF-8, trocando **Requisições** (`id`, `method`, `params`), **Respostas** (`id`, `result` ou `error`) e **Notificações** (sem `id`), reservando o campo `_meta` em `params` e `result` para propagar metadados operacionais e cabeçalhos de rastreamento distribuído. O ciclo de sessão compreende três etapas: (1) **Inicialização (*Handshake*)**, via `initialize` e `notifications/initialized`; (2) **Descoberta e Operação**, via listagem (`tools/list`, `resources/list`) e execução (`tools/call`, `resources/read`); e (3) **Encerramento**, com o fechamento do canal (Hou et al., 2025).

Na camada de transporte, operam dois mecanismos principais (Guo et al., 2025):

- **Entrada e Saída Padrão (*stdio*):** Utilizado localmente, onde o *host* instancia o servidor como subprocesso filho e troca mensagens via `stdin`/`stdout`. Não possui autenticação nativa e faz o servidor herdar todas as permissões de disco, variáveis de ambiente e rede do usuário local (Bühler et al., 2026);
- **HTTP Transmissível (*Streamable HTTP*):** Substituiu o transporte legado *Server-Sent Events* (SSE), unificando requisições POST e GET em um único endpoint HTTP com suporte a fluxos contínuos, identificação de sessão (`Mcp-Session-Id`) e autenticação padronizada para servidores remotos.

2.2 Superfície de Ataque e Modelagem de Ameaças em Ecossistemas MCP

Embora a padronização trazida pelo MCP tenha simplificado a conexão entre agentes e sistemas externos, sua arquitetura original priorizou a facilidade de integração em detrimento de barreiras nativas de segurança. Ao delegar a execução de rotinas diretamente a processos externos locais ou remotos sem um mecanismo intermediário de validação, o ecossistema passou a combinar fragilidades clássicas de software com novos vetores de manipulação cognitiva próprios de modelos de linguagem. Dessa forma, uma simples leitura de dados externos pode converter-se no ponto de partida para o comprometimento do sistema hospedeiro.

Para examinar essa superfície de risco de maneira estruturada, esta seção discute inicialmente, na Subseção 2.2.1, como o paradigma de confiança implícita e as falhas ao

longo das fases do ciclo de vida do servidor expõem o ambiente operacional. Na sequência, a Subseção 2.2.2 sistematiza a taxonomia de ataques nas camadas cognitiva, protocolar e de infraestrutura, enquanto a Subseção 2.2.3 fundamenta os fenômenos da lacuna semântica e da deriva de privilégios que tornam insuficientes os mecanismos tradicionais de proteção.

2.2.1 O Paradigma de Confiança Implícita e o Ciclo de Vida do Servidor

Diferentemente de sistemas modernos que isolam aplicativos em *sandboxes* com manifestos estritos de permissão (como no Android ou macOS), o MCP opera sob **confiança implícita**: concluída a inicialização, o protocolo assume que o servidor é benigno e que suas descrições são fidedignas. Isso gera três fragilidades estruturais: (i) ausência de manifesto nativo de permissões de sistema operacional; (ii) visibilidade plana de todas as ferramentas na sessão, misturando consultas e mutações; e (iii) execução imediata de chamadas `tools/call` sem validação determinística de intenção (Bühler et al., 2026; Singh e Madisetti, 2026).

Para rastrear a origem dessas falhas, Hou et al. (Hou et al., 2025) dividiram o ciclo de vida do servidor MCP em quatro fases e 16 atividades: (1) **Criação** (definição de metadados, declaração de capacidades, implementação de código e comandos de atalho `npx`/`uvx`); (2) **Implantação** (publicação de versão, execução de auto-instaladores, gravação de chaves em JSON local e registro no *host*); (3) **Operação** (análise de intenção pelo LLM, acesso a recursos externos, invocação `tools/call` e gestão de contexto); e (4) **Manutenção** (controle de versão, mudanças de configuração, auditoria de acesso e auditoria de logs), cuja ausência de controles permite que atualizações silenciosas introduzam cargas maliciosas mantendo privilégios antigos.

2.2.2 Taxonomia de Vetores de Ataque: Camadas Cognitiva, Protocolar e de Infraestrutura

Como o MCP funciona como um barramento *plug-and-play*, ele acumula vulnerabilidades de todos os subsistemas conectados. Jang e Lee (Jang e Lee, 2026) e Hou et al. (Hou et al., 2025) classificam esses vetores em três camadas:

1. **Ameaças Inerentes ao Modelo de Linguagem (LIT):** Decorrem da incapacidade dos transformadores de separar instruções do sistema de dados externos não confiáveis:

    - *Injeção Indireta de Prompt:* Comandos ocultos em páginas, documentos ou tíquetes lidos pelo agente sequestram seu objetivo original e o forçam a acionar ferramentas conectadas (Singh e Madisetti, 2026);

- *Vazamento de Contexto e Envenenamento de Recursos:* Indução do modelo a exfiltrar dados privados do contexto via argumentos de ferramentas ou corrupção de fontes lidas em `resources/read`.

2. **Ameaças Específicas do Protocolo MCP (MST):** Exploram a descoberta dinâmica (`tools/list`), a leitura semântica de esquemas JSON e a coexistência de múltiplos servidores:

   - *Envenenamento de Ferramentas (Tool Poisoning):* Inserção de instruções ocultas no campo `description` ou no esquema de argumentos da ferramenta. Enquanto o usuário vê apenas o nome da função na interface, o LLM lê a descrição completa e é induzido a ler arquivos sensíveis (como `~/.ssh/id_rsa` ou `.env`) e enviá-los como parâmetro antes de executar a rotina, sem alterar o binário do servidor (Hasan et al., 2026);
   - *Atualização Fraudulenta (Rug Pull):* Servidores publicados inicialmente de forma benigna que, aproveitando execuções sem fixação de versão, introduzem código malicioso em atualizações posteriores;
   - *Sombreamento Cruzado (Tool Shadowing) e Ataques Fantoche:* Um servidor malicioso injeta regras no contexto que manipulam como o agente invoca ferramentas de outros servidores legítimos ativos na mesma sessão;
   - *Conflito de Nomes, Typosquatting e Manipulação de Preferência (PMA e GAPMA):* Registro de pacotes com grafia semelhante aos oficiais ou uso de gatilhos otimizados nas descrições para forçar o LLM a selecionar sempre a ferramenta do atacante;
   - *Abuso de Encadeamento de Ferramentas:* Composição autônoma de ferramentas individualmente seguras (ex.: leitura de arquivo seguida de chamada HTTP) para formar um canal completo de exfiltração (Singh e Madisetti, 2026).

3. **Ameaças Herdadas da Cibersegurança (CIT):** Vulnerabilidades clássicas de software e rede amplificadas pela automação:

   - *Injeção de Comandos, Path Traversal e Execução Remota (RCE):* Passagem direta de parâmetros de `tools/call` para chamadas de sistema (`exec`/`spawn`) sem sanitização, como na falha crítica CVE-2025-49596 no utilitário oficial *MCP Inspector* (Hasan et al., 2026);
   - *Roubo de Credenciais:* Leitura de chaves de API salvas em texto puro em arquivos locais previsíveis (`claude_desktop_config.json` e `~/.cursor/mcp.json`) ou expostas no código-fonte;

- *Escape de Sandbox, Sequestro SSE e Envenenamento de DNS:* Sondagem de portas locais, interceptação de fluxos SSE sem criptografia ou adulteração da resolução de nomes (`/etc/hosts` e DNS) para redirecionar consultas de ferramentas a servidores falsos (Jang e Lee, 2026).

2.2.3 A Lacuna Semântica e a Deriva de Privilégios

A insuficiência das defesas convencionais em ecossistemas MCP explica-se por dois conceitos centrais: a **lacuna semântica** e a **deriva de privilégios**. A **Lacuna Semântica** (*Semantic Gap*), formalizada por Bühler et al. (Bühler et al., 2026), define a distância de abstração entre controles de sistema operacional e ataques ao protocolo MCP. Contêineres Docker e regras `iptables` restringem o acesso a diretórios físicos e endereços IP; contudo, quando um servidor MCP precisa conectar-se legitimamente a `api.github.com` na porta 443, o filtro de rede deve permitir esse tráfego. Dentro desse canal autorizado, o sistema operacional é cego à semântica das mensagens JSON-RPC 2.0, não distinguindo uma consulta benigna (`get_issue`) de um comando destrutivo (`delete_branch` ou `force_push`).

Por sua vez, a **Deriva de Privilégios** (*Privilege Drift*), descrita por Singh e Madisetti (Singh e Madisetti, 2026), ocorre em fluxos de múltiplas etapas: a leitura sucessiva de dados externos não confiáveis contamina progressivamente o contexto do modelo, comprometendo seu alinhamento inicial e levando-o a acionar ferramentas de escrita ou exclusão habilitadas na mesma sessão. Além disso, delegar a decisão de segurança a outro LLM em *runtime* mostra-se ineficaz, pois modelos probabilísticos carecem de determinismo formal, adicionam alta latência e falham em quase metade das decisões de controle de acesso (Zhang et al., 2025).

2.3 Governança em Tempo de Execução e Controle de Acesso Baseado em Papéis

Diante da incapacidade dos filtros de sistema operacional de enxergar a intenção das chamadas JSON-RPC e da imprevisibilidade de delegar decisões de segurança ao próprio modelo de linguagem, torna-se indispensável introduzir uma camada de controle determinística diretamente no canal de comunicação do protocolo. Ao mediar cada requisição antes que ela alcance o servidor de destino, é possível restringir as capacidades visíveis ao agente, validar os argumentos das chamadas contra regras estritas e impedir que operações de escrita ou exclusão sejam disparadas sem autorização explícita.

A construção teórica dessa camada de governança organiza-se em três frentes complementares: a Subseção 2.3.1 introduz o Princípio do Menor Privilégio aplicado a agentes e a arquitetura de Monitores de Referência dividida entre pontos de imposição e de decisão; a Subseção 2.3.2 formaliza o modelo matemático de Controle de Acesso Baseado em

Papéis (RBAC) e os predicados de guarda sobre argumentos; e a Subseção 2.3.3 detalha a máquina de estados de sessão em quatro níveis de segurança, a elevação temporária supervisionada por humanos e o fenômeno de moldagem comportamental.

2.3.1 Princípio do Menor Privilégio e Monitores de Referência

O **Princípio do Menor Privilégio** estabelece que cada agente autônomo deve operar apenas com as permissões estritamente necessárias para concluir sua tarefa atual e pelo menor tempo possível. Para impor esse princípio sem modificar o código-fonte de servidores de terceiros, adota-se o padrão clássico de **Monitor de Referência** (mediação completa, resistência à adulteração e verificabilidade), desacoplado em dois módulos sobre o fluxo JSON-RPC 2.0 (Singh e Madisetti, 2026):

- **Ponto de Imposição de Política (*Policy Enforcement Point* - PEP):** Proxy transparente entre cliente e servidor que intercepta tanto as respostas de descoberta (`tools/list`, ocultando ferramentas fora do escopo autorizado) quanto as chamadas de execução (`tools/call` e `resources/read`);
- **Ponto de Decisão de Política (*Policy Decision Point* - PDP):** Motor determinístico que avalia a identidade da sessão, o papel ativo, o método, o efeito operacional e os argumentos contra regras formais, retornando instantaneamente o veredito (*Permitir*, *Bloquear*, *Rebaixar* ou *Solicitar Elevação HITL*) com latência imperceptível.

2.3.2 Modelo Formal de Controle de RBAC e Taxonomias de Permissão

O **Controle de Acesso Baseado em Papéis** (RBAC) desacopla as sessões de agentes das operações brutas por meio de papéis funcionais. Adaptando a formalização de Zhang et al. (Zhang et al., 2025) e Singh e Madisetti (Singh e Madisetti, 2026) ao protocolo MCP, o modelo de autorização é definido pelos seguintes conjuntos:

- S: conjunto de sessões ativas de agentes conectados ao gateway;
- R: conjunto finito de papéis organizacionais (ex.: *Agente Restrito*, *Auditor*, *Operador* e *Administrador*);
- $O = T \cup U$: conjunto de objetos protegidos, unindo ferramentas T (`tools`) e recursos U (`resources`);
- $E = \{\texttt{read}, \texttt{write}, \texttt{delete}\}$: conjunto de efeitos operacionais semânticos, separando consultas sem efeitos colaterais (`read`), mutações controladas (`write`) e ações destrutivas (`delete`);

- C: conjunto de predicados de verificação condicional sobre os argumentos `args` (listas de caminhos e domínios permitidos e limites de taxa) (Bühler et al., 2026);
- $P \subseteq O \times E \times C$: conjunto de permissões qualificadas $p = (o, e, c)$, aliado às relações de atribuição de sessão a papel ($UA \subseteq S \times R$) e de permissão a papel ($PA \subseteq R \times P$).

Dada uma requisição $req = (s, o, args)$ cujo objeto o possui efeito e, o PDP autoriza a execução se, e somente se, a condição da Equação 2.1 for satisfeita:

$$\text{Auth}(s, o, args) \iff \exists r \in R : (s, r) \in UA \ \wedge \ (r, (o, e, c)) \in PA \ \wedge \ c(args) \qquad (2.1)$$

Essa estrutura impõe a tríade formal **Papel–Operação–Verificação**: papéis básicos permanecem restritos a consultas ($e =$ `read`), papéis operacionais executam escritas ($e =$ `write`) confinadas pelos predicados $c(args)$, e ações destrutivas ($e =$ `delete`) exigem papéis administrativos aliados à supervisão humana.

2.3.3 Máquina de Estados de Sessão e Elevação Supervisionada

Para conter a deriva de privilégios em sessões com permissão nominal de escrita, Singh e Madisetti (Singh e Madisetti, 2026) modelaram a governança como uma **Máquina de Estados de Segurança em Nível de Sessão** baseada em três controles: (1) **Acesso Escopado**, exibindo em `tools/list` somente as ferramentas disponíveis ao estado atual; (2) **Padrão Somente-Leitura**, mantendo a sessão por padrão em modo restrito de consulta e aplicando bloqueio determinístico ou rebaixamento seguro a chamadas `write`/`delete`; e (3) **Elevação Supervisionada com Janela Temporal**, na qual mutações legítimas transitam a sessão para `PendingApproval` e exigem confirmação humana (*Human-in-the-Loop* - HITL), abrindo uma janela temporária de privilégio que reverte automaticamente ao estado somente-leitura após a conclusão.

Operando em quatro níveis cumulativos (**L0**: Baseline Aberto; **L1**: Acesso Escopado; **L2**: Padrão Somente-Leitura; e **L3**: Elevação Supervisionada HITL), essa máquina induz a **Moldagem Comportamental** (*Behavioral Shaping*): ao receber recusas determinísticas para ações mutantes e enxergar um catálogo restrito, o próprio LLM ajusta seu planejamento interno (ReAct) para estratégias conservadoras e seguras.

2.4 Auditoria Forense Imutável e Observabilidade Distribuída em Tempo Real

O bloqueio preventivo de ações não autorizadas em tempo de execução resolve apenas parte do desafio de governança em sistemas autônomos. Em ambientes críticos, é igualmente necessário comprovar matematicamente que o histórico de decisões e chamadas não foi adulterado após um incidente, além de monitorar continuamente o comportamento

das sessões e o impacto de latência introduzido pela camada de segurança. Sem mecanismos de integridade criptográfica e telemetria padronizada, os registros locais tornam-se alvos fáceis de manipulação e a operação do sistema permanece opaca para auditores e administradores.

Para suprir essa lacuna de visibilidade e perícia, a Subseção 2.4.1 apresenta a estruturação de registros de auditoria encadeados criptograficamente por meio da função de resumo SHA-256. Em complemento, a Subseção 2.4.2 discute as garantias formais de integridade fim a fim, rastreabilidade causal em múltiplos passos e não-repúdio operacional exigidas por normas internacionais de segurança, enquanto a Subseção 2.4.3 fundamenta a instrumentação distribuída em tempo real utilizando o padrão OpenTelemetry e a propagação de contexto W3C.

2.4.1 Registro Estruturado e Encadeamento Criptográfico de Resumos

Os logs nativos de servidores MCP em `stderr` são despadronizados, carecem de visão das políticas do *host* e podem ser editados ou apagados caso o processo seja comprometido. Para garantir validade pericial, emprega-se o **Registro Estruturado à Prova de Adulteração** (*Tamper-Evident Logging*) em formato *JSON Lines* (JSONL) com encadeamento criptográfico **SHA-256** (Jang e Lee, 2026). Cada evento $i \geq 1$ registra o índice sequencial (i), carimbo de tempo UTC ISO 8601 (T_i), identificador de sessão e rastro (ID_i), papel ativo (R_i), método/ferramenta (M_i), parâmetros normalizados (P_i), decisão do PDP (D_i) e status/latência (S_i). Partindo de um bloco gênese $H_0 = 0^{256}$, o resumo H_i incorpora recursivamente o resumo anterior:

$$H_i = \text{SHA-256}\left(H_{i-1} \parallel i \parallel T_i \parallel ID_i \parallel R_i \parallel M_i \parallel P_i \parallel D_i \parallel S_i\right) \tag{2.2}$$

onde $\parallel$ denota a concatenação após serialização JSON. Pela resistência à colisão da função SHA-256, qualquer edição, exclusão ou inserção retroativa em um registro k produz $H'_k \neq H_k$ e invalida em cascata todos os resumos subsequentes $(H_{k+1}, \ldots, H_n)$.

2.4.2 Garantias de Integridade, Rastreabilidade Causal e Não-Repúdio

A trilha encadeada assegura três propriedades exigidas por normas como NIST AI RMF 1.0, NIST CSF 2.0, CIS Controls v8 e ISO/IEC 27001 (Jang e Lee, 2026): (1) **Integridade Verificável de Fim a Fim**, permitindo auditar a cadeia desde H_0 em tempo linear $O(n)$; (2) **Rastreabilidade Causal Multi-Etapas**, reconstruindo a ordem cronológica da sessão ID_i para identificar qual leitura externa contaminou o contexto do agente antes de uma tentativa de mutação bloqueada pelo PDP; e (3) **Não-Repúdio**

Operacional e Humano, vinculando irrevogavelmente chamadas do agente e aprovações humanas HITL às suas identidades e janelas de tempo.

2.4.3 Telemetria Padronizada com OpenTelemetry e Rastreamento Distribuído

Para evitar subsistemas proprietários de amostragem, adota-se na camada de governança o padrão aberto **OpenTelemetry** (OTel) estruturado em três sinais: (i) **Rastreamentos Distribuídos (*Traces*)**, que propagam os cabeçalhos `traceparent` (`trace-id` de 128 bits e `parent-id` de 64 bits) e `tracestate` do padrão *W3C Trace Context* dentro do objeto reservado `_meta` do JSON-RPC 2.0, instanciando segmentos para medir isoladamente a avaliação do gateway; (ii) **Métricas Quantitativas**, com contadores monotônicos de decisões RBAC, histogramas de latência ($p50$, $p95$, $p99$) e medidores de sessões ativas (L0–L3); e (iii) **Registros Correlacionados**, vinculando o `trace_id` a cada linha da trilha forense JSONL (Equação 2.2).

2.5 Trabalhos Relacionados

A rápida adoção do protocolo MCP motivou uma série de investigações recentes voltadas a compreender a dimensão real de seu ecossistema, catalogar suas fragilidades de segurança e propor mecanismos iniciais de contenção. Analisar criticamente o alcance e as limitações dessas abordagens na literatura recente permite evidenciar por que soluções isoladas de análise estática ou de confinamento ao nível do sistema operacional mostram-se insuficientes quando desacompanhadas de governança de sessão, trilha forense imutável e observabilidade em tempo real.

Para oferecer uma visão integrada do estado da arte, a revisão dos sete artigos de base deste trabalho está organizada em quatro partes: a Subseção 2.5.1 examina os levantamentos empíricos de larga escala sobre a qualidade e as vulnerabilidades dos servidores públicos; a Subseção 2.5.2 discute os estudos de modelagem de ameaças ao longo do ciclo de vida do protocolo e frameworks hierárquicos de defesa; a Subseção 2.5.3 avalia as arquiteturas de isolamento em contêineres, controle de acesso em tempo de execução e mineração formal de regras RBAC; e a Subseção 2.5.4 consolida a comparação direta entre as obras e destaca os métodos que inspiram o artefato proposto nesta monografia.

2.5.1 Estudos Empíricos de Medição, Qualidade e Segurança do Ecossistema MCP

Hasan et al. (Hasan et al., 2026) conduziram o primeiro estudo empírico sobre a segurança e a manutenibilidade de servidores MCP de código aberto, combinando análise estática (*SonarQube*) em 583 repositórios representativos com varredura dinâmica

(`mcp-scan`) sobre 73 servidores no *Claude Desktop*. Os autores identificaram vulnerabilidades clássicas de código em 7,2% dos projetos (como credenciais expostas e *Path Traversal*), *Tool Poisoning* ativo nas descrições de 5,5% dos servidores testados e problemas estruturais de manutenibilidade em 66,0% da amostra, evidenciando a ausência de validação preventiva nos repositórios comunitários.

De forma complementar, Guo et al. (Guo et al., 2025) mapearam seis diretórios públicos com a ferramenta **MCPCrawler**, filtrando 8.060 servidores válidos e 341 clientes a partir de 16.950 registros brutos (dos quais 50,9% eram links inativos). A análise estática de árvores sintáticas revelou que 50,8% dos servidores (4.095 projetos) invocam diretamente APIs sensíveis de rede, execução de código (`eval`/`exec`), comandos de sistema ou arquivos locais sem confinamento, enquanto 99,4% dos clientes dependem do transporte local *stdio* sem autenticação nativa.

2.5.2 Modelagem de Ameaças, Ciclo de Vida e Frameworks Hierárquicos de Defesa

Hou et al. (Hou et al., 2025) analisaram 26 diretórios comunitários e demonstraram que auto-instaladores populares executam pacotes sem verificação criptográfica de integridade. A partir do modelo de ciclo de vida já discutido na Seção 2.2, os autores validaram experimentalmente 16 cenários de ataque distribuídos entre desenvolvedores maliciosos, atacantes externos, usuários locais e falhas de manutenção, destacando a falta de auditoria contínua e de expiração de privilégios após atualizações.

Motivados por ataques reais de atualização fraudulenta (*Rug Pull*), Jang e Lee (Jang e Lee, 2026) propuseram o **MCP Security Framework**, sistematizando 28 ameaças sobre a Tríade CIA. Em bancada experimental com máquinas virtuais Linux, demonstraram que o envenenamento de DNS e do arquivo `/etc/hosts` compromete a resposta semântica do agente sem acionar filtros de texto, propondo o **MCP Firewall** (baseado em verificação estática via SHA-256 e validação cruzada de DNS) alinhado às diretrizes do NIST e do CIS Controls.

2.5.3 Arquiteturas de Isolamento, Controle de Acesso em Tempo de Execução e Práticas Formais de RBAC

Bühler et al. (Bühler et al., 2026) apresentaram o framework **AgentBound**, composto pelo manifesto declarativo `AgentManifest`, pelo gerador estático `AgentManifestGen` (80,9% de precisão e 100% de *recall*) e pelo motor de isolamento em contêineres Docker e `iptables` (`AgentBox`). Avaliado nos 296 servidores mais populares do *PulseMCP*, o mecanismo bloqueou 100% dos ataques ao sistema operacional com sobrecarga inferior a 1%, mas demonstrou que o isolamento de infraestrutura não resolve a lacuna semântica dentro de canais de rede já autorizados.

Para atuar diretamente sobre o tráfego JSON-RPC, Singh e Madisetti (Singh e Madisetti, 2026) desenvolveram o interceptador **MCP-Secure**, regido pela máquina de estados em quatro níveis (L0 a L3) detalhada na Seção 2.3. Em 1.080 execuções experimentais com os modelos GPT-5, Claude Sonnet 4 e Grok-4, a contenção de chamadas indevidas subiu de 0,00% no nível L0 para 75,30% no L1, 100,00% no L2 e 97,50% no L3, com latência mediana entre 0,34 ms e 0,57 ms e indução de moldagem comportamental, embora sem suporte a auditoria criptográfica encadeada ou telemetria OpenTelemetry.

Por fim, Zhang et al. (Zhang et al., 2025) propuseram o sistema **ACFIX**, que minerou 344.251 contratos inteligentes para extrair uma taxonomia de 49 práticas formais de RBAC nos perfis *Admin*, *Operator* e *User*. Na validação sobre 118 vulnerabilidades reais de controle de acesso, o uso da taxonomia determinística de RBAC elevou a taxa de correção para 94,92%, superando amplamente o uso isolado do modelo GPT-4 (52,54%) e comprovando a necessidade de manter decisões de permissão fora do raciocínio probabilístico do LLM.

2.5.4 Discussão Comparativa e Síntese dos Trabalhos Relacionados

A análise conjunta das obras revisadas demonstra que as pesquisas atuais abordam o problema de segurança de forma fragmentada: enquanto levantamentos empíricos diagnosticam falhas de código e de protocolo (Hasan et al., 2026; Guo et al., 2025; Hou et al., 2025) e defesas de infraestrutura restringem o acesso ao sistema operacional ou à resolução de rede (Jang e Lee, 2026; Bühler et al., 2026), as abordagens de mediação de sessão e de RBAC (Singh e Madisetti, 2026; Zhang et al., 2025) ainda carecem de trilhas forenses imutáveis encadeadas em tempo de execução e de observabilidade distribuída padronizada. A Tabela 2.1 consolida a comparação direta entre os sete artigos de base e o artefato proposto nesta monografia (*Governance Gateway*), destacando os métodos absorvidos de cada referência para compor uma camada unificada de governança, auditoria SHA-256 e telemetria OpenTelemetry.

Tabela 2.1: Comparação entre os trabalhos relacionados da literatura e a proposta desta monografia.

Trabalho	**Foco**	**Camada**	**RBAC**	**Auditoria**	**Telemetria**
Hasan et al. (Hasan et al., 2026) (ACM TOSEM)	Auditoria híbrida de 1.899 servidores MCP e envenenamento.	Repositório e Execução	Não (Diagnóstico)	Não	Não
Guo et al. (Guo et al., 2025) (arXiv / NTU)	Medição de 6 mercados (8.060 servidores) e APIs sensíveis.	Ecossistema e Código-Fonte	Não (Diagnóstico)	Não	Não
Hou et al. (Hou et al., 2025) (arXiv / HUST)	Ciclo de vida em 4 fases, 16 ataques e provas de conceito.	Arquitetura de Protocolo MCP	Conceitual	Recomendado na Fase 4	Não
Jang e Lee (Jang e Lee, 2026) (IEEE Access)	Taxonomia de 28 ameaças, modelo de risco e *MCP Firewall*.	Rede / Host	Parcial (Diretrizes DS1)	Resumo SHA-256 estático local	Conceitual (Monitoramento)
Bühler et al. (Bühler et al., 2026) (ACM FSE)	Controle declarativo (`AgentManifest`) e isolamento (`AgentBox`).	Sistema Operacional / Contêiner	Sim (Capacidades de SO)	Não	Não
Singh e Madisetti (Singh e Madisetti, 2026) (IEEE OJCS)	Mediação em *runtime* (*MCP-Secure*), níveis L0–L3 e HITL.	Barramento JSON-RPC	Sim (Escopo e Efeitos L0–L3)	Não (Registros voláteis)	Não
Zhang et al. (Zhang et al., 2025) (IEEE TSE)	Mineração de 49 práticas RBAC (*ACFIX*) e grafo ACG.	Código Estático	Sim (Taxonomia RBAC formal)	Não (Eventos em código)	Não
Proposta (Este Trabalho)	**Camada integrada de governança, auditoria e telemetria.**	**Barramento JSON-RPC (Proxy)**	**Sim (RBAC determinístico + L0–L3 + HITL)**	**Sim (JSONL encadeado via SHA-256)**	**Sim (Nativo OpenTelemetry + W3C)**

Fonte: Elaborado pelo autor (2026).

Capítulo 3

Modelo Conceitual do Artefato

No contexto do problema geral abordado, ecossistemas baseados no *Model Context Protocol* operam sob um modelo de confiança implícita no qual o agente autônomo no *host* interage diretamente com servidores de ferramentas locais ou remotos sem mediação determinística em *runtime* (Bühler et al., 2026; Singh e Madisetti, 2026). Essa ausência de fronteira de governança expõe o ambiente a ataques de *Tool Poisoning*, injeção indireta de instruções, deriva de privilégios em fluxos multi-etapas e adulteração de históricos operacionais (Hasan et al., 2026; Hou et al., 2025; Jang e Lee, 2026). Para solucionar esse problema sem exigir modificações no código-fonte dos clientes ou dos servidores MCP legados, o modelo conceitual posiciona o *Governance Gateway* como uma camada intermediária transparente de interceptação (*proxy*) no barramento JSON-RPC 2.0.

Este capítulo apresenta o modelo conceitual do artefato proposto nesta monografia, denominado *Governance Gateway*, concebido para solucionar o problema geral formulado na Seção 1.2 e demonstrar a hipótese geral estabelecida na Seção 1.3. Enquanto o Capítulo 2 fundamentou a base teórica e analisou o estado da arte (Hasan et al., 2026; Singh e Madisetti, 2026; Guo et al., 2025; Hou et al., 2025; Jang e Lee, 2026; Bühler et al., 2026; Zhang et al., 2025), o presente capítulo especifica os elementos arquiteturais, os módulos de processamento, as estruturas de dados e os vínculos de dependência que constituem a solução projetada.

A arquitetura geral organiza-se em quatro módulos funcionais interdependentes, nos quais a saída processada por um segmento alimenta a decisão ou a persistência dos módulos seguintes, respondendo diretamente aos subproblemas e hipóteses específicas da pesquisa:

1. **Módulo de Interceptação e Roteamento de Sessão:** Atua na fronteira entre o *MCP Client* e o *MCP Server*, capturando o tráfego bruto nos canais *stdio* e *Streamable HTTP*, decodificando os quadros JSON-RPC 2.0 para extrair o contexto estruturado da chamada e impondo a filtragem de catálogos e o bloqueio de pacotes não autorizados;

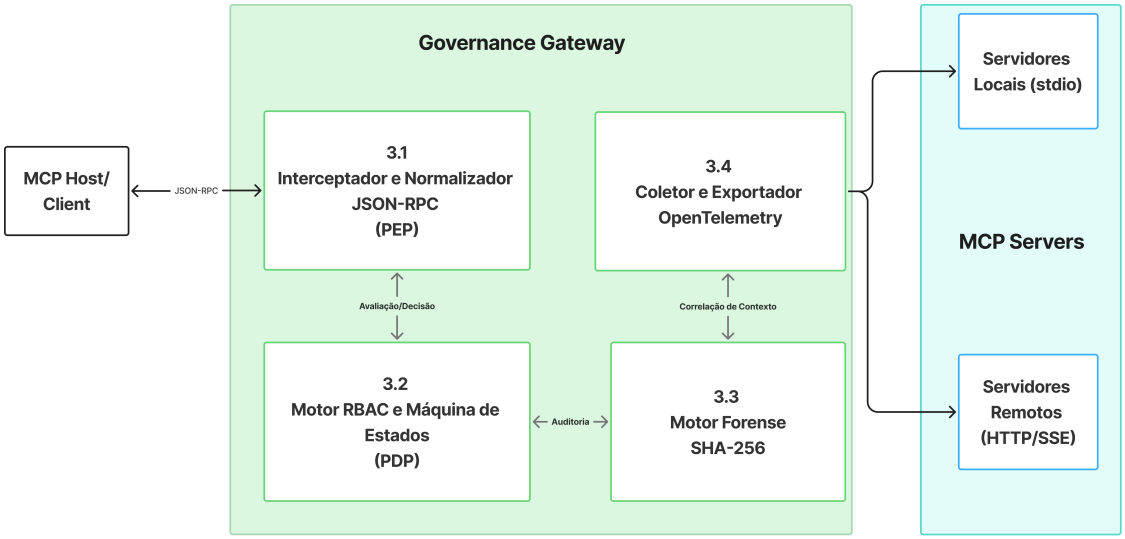
2. **Módulo de Governança Determinística, RBAC e Máquina de Estados de Sessão:** Recebe o contexto normalizado da chamada e atua como Ponto de Decisão de Política (PDP), combinando uma matriz formal de RBAC (Zhang et al., 2025) com uma máquina de estados de sessão em quatro níveis (L0 a L3) (Singh e Madisetti, 2026) e elevação supervisionada (HITL) para emitir um veredito determinístico;

3. **Módulo de Auditoria Forense Imutável e Verificação Criptográfica:** Consome os metadados da requisição e o veredito do PDP para construir registros estruturados em formato *JSON Lines* encadeados recursivamente via SHA-256 (Jang e Lee, 2026), assegurando integridade fim a fim, rastreabilidade causal e não-repúdio;

4. **Módulo de Observabilidade Distribuída e Telemetria em *Runtime*:** Opera transversalmente extraindo e injetando cabeçalhos *W3C Trace Context* no campo reservado `_meta`, cronometrando os estágios internos em microssegundos, correlacionando identificadores de *trace* com os registros SHA-256 e exportando telemetria no padrão OpenTelemetry.

A Figura 3.1 apresenta o diagrama da arquitetura conceitual geral do *Governance Gateway*, ilustrando a disposição dos quatro módulos e seus vínculos. Por fim, a Seção 3.5 consolida a integração fim a fim desses processos e explicita o mapeamento com os artigos científicos inspiradores revisados no Capítulo 2.

Figura 3.1: Arquitetura Conceitual Geral do *Governance Gateway* e Vínculos de Dependência entre Módulos.



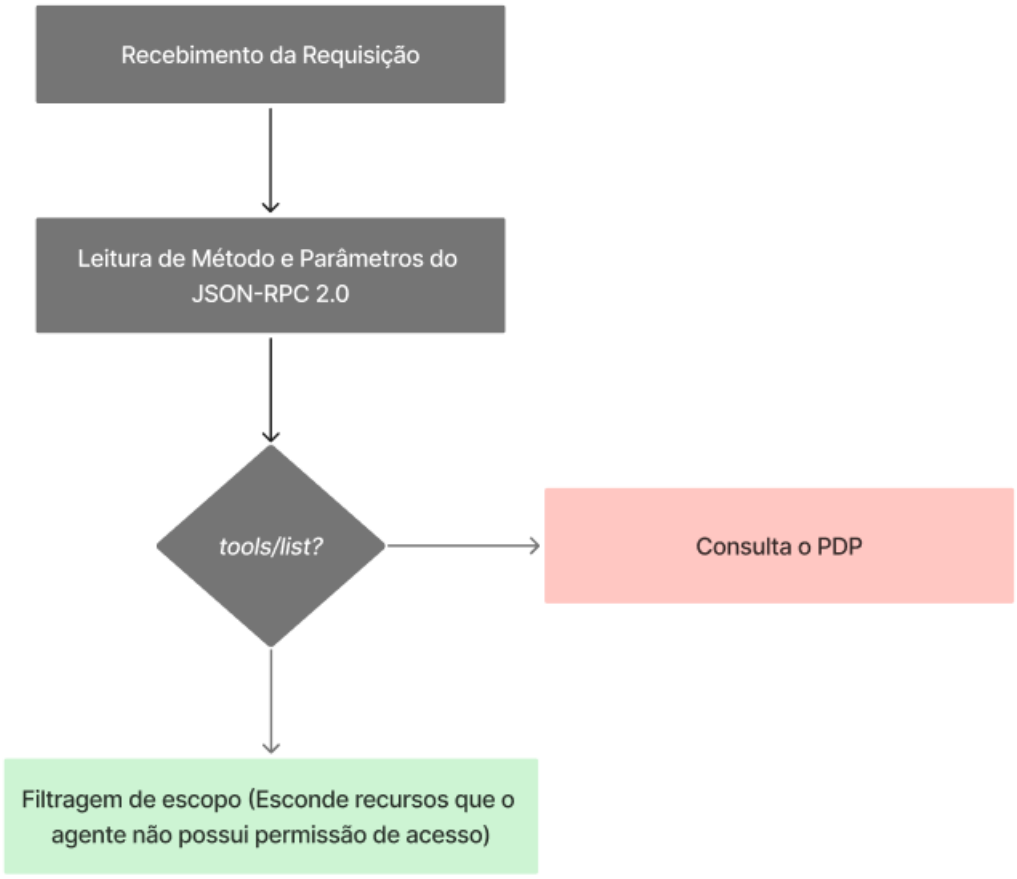
Fonte: Elaborado pelo autor (2026).

3.1 Módulo de Interceptação e Roteamento de Sessão

O primeiro requisito para aplicar governança no ecossistema MCP é interceptar a comunicação entre o agente e as ferramentas sem precisar alterar o código-fonte do *host* ou dos servidores comunitários (Hasan et al., 2026; Guo et al., 2025). O Módulo de Interceptação Transparente e Roteamento de Sessão cumpre esse papel funcionando como o Ponto de Imposição de Política (PEP) do gateway (Singh e Madisetti, 2026).

Na prática, este módulo posiciona-se como um *proxy* intermediário: ele recebe as mensagens enviadas nos canais de transporte, extrai os dados principais da requisição JSON-RPC 2.0, consulta o Módulo de Governança (Seção 3.2) e aplica a decisão retornada (encaminhar, filtrar ou bloquear). Sua operação divide-se em três submódulos: Adaptação dos Canais de Transporte (Subseção 3.1.1), Decodificação e Extração de Contexto JSON-RPC (Subseção 3.1.2) e Filtragem de Catálogo e Imposição de Política (Subseção 3.1.3), conforme ilustrado na Figura 3.2.

Figura 3.2: Fluxograma do Módulo de Interceptação e Imposição de Política (PEP).



Fonte: Elaborado pelo autor (2026).

3.1.1 Adaptação dos Canais de Transporte

Conforme demonstrado por Guo et al. (Guo et al., 2025), 99,4% dos servidores MCP operam localmente via entrada e saída padrão (*stdio*), enquanto servidores remotos utilizam *Streamable HTTP*. Para suportar ambos os cenários sem modificar os sistemas existentes, este submódulo opera em dois modos transparentes:

- **Interceptação Local (*stdio*):** No arquivo de configuração do *host* (como `claude_desktop_config.json`), o comando de inicialização do servidor é substituído pelo executável do gateway, que recebe o comando original (`npx`, `uvx` ou `python`) como argumento (Hou et al., 2025; Bühler et al., 2026). O gateway inicia o servidor real como um subprocesso filho e redireciona seus fluxos `stdin` e `stdout`, inspecionando todas as mensagens trocadas;
- **Interceptação Remota (*Streamable HTTP*):** Para servidores em rede, o submódulo atua como um *reverse proxy* HTTP que recebe as requisições do cliente, preserva o cabeçalho de sessão (`Mcp-Session-Id`) e repassa ao servidor remoto apenas as chamadas autorizadas (Singh e Madisetti, 2026).

3.1.2 Decodificação e Extração de Contexto JSON-RPC

Uma vez capturada a mensagem no canal de transporte, este submódulo valida a estrutura JSON-RPC 2.0 e monta em memória o **Contexto de Interceptação** ($\mathcal{C}_{\text{req}}$) que será enviado aos demais módulos. Esse processo resume-se à extração de três informações principais:

1. **Operação e Alvo:** Identifica o método invocado (`initialize`, `tools/list`, `tools/call` ou `resources/read`), o nome da ferramenta ou recurso (`params.name` ou `params.uri`) e seus argumentos (`params.arguments`);
2. **Metadados de Rastreamento (`_meta`):** Separa o campo reservado `params._meta` dos argumentos da ferramenta, extraindo o cabeçalho `traceparent` utilizado pelo Módulo de Observabilidade (Seção 3.4);
3. **Vínculo de Sessão e Tempo:** Associa a mensagem ao identificador da sessão ativa ($s \in S$) e registra o instante exato de chegada (t_{in}) para medição da latência interna do gateway.

3.1.3 Filtragem de Catálogo e Imposição de Política (PEP)

O terceiro submódulo executa de fato a filtragem e o controle sobre o tráfego (Singh e Madisetti, 2026; Hasan et al., 2026). Quando o servidor responde a uma consulta de descoberta (`tools/list`), o PEP realiza a **filtragem de escopo**, ocultando ferramentas

que não pertencem ao papel do agente e a **higienização de descrições**, removendo instruções maliciosas ocultas nos campos `description` (*Tool Poisoning*) (Hasan et al., 2026; Hou et al., 2025) antes que cheguem ao modelo de linguagem.

Por sua vez, quando o cliente envia uma chamada de execução (`tools/call` ou `resources/read`), o PEP consulta o Módulo 3.2 (PDP) e aplica uma das quatro ações possíveis:

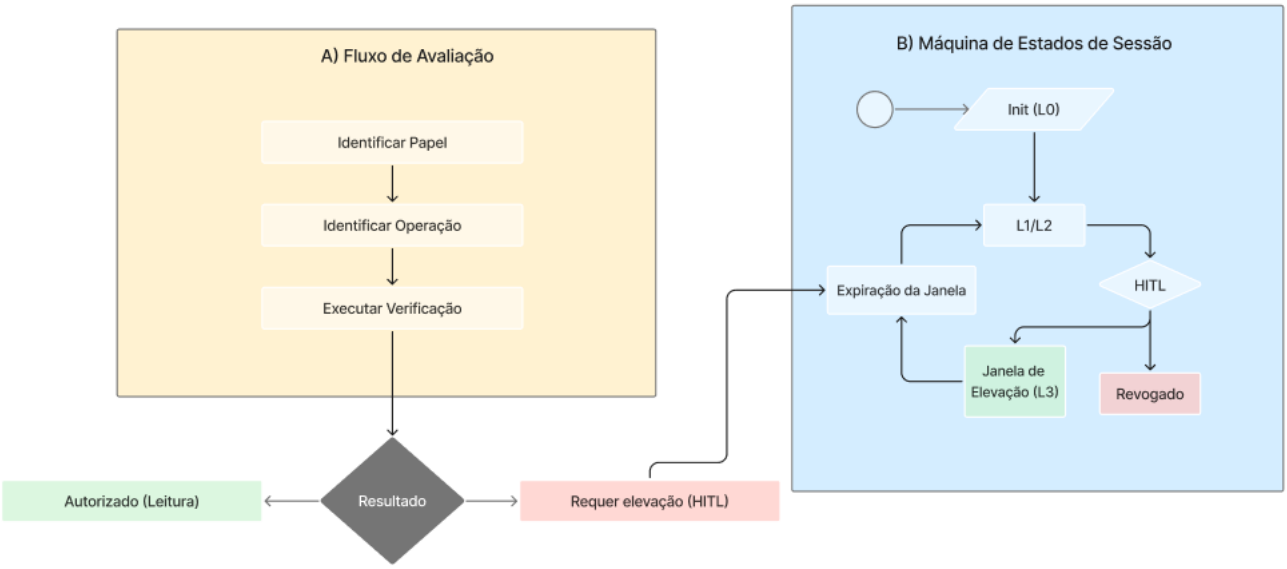
- `ALLOW`: Encaminha a requisição normalmente ao servidor MCP;
- `DENY`: Bloqueia a chamada e retorna imediatamente ao cliente um erro JSON-RPC padronizado de violação de política;
- `DOWNGRADE`: Substitui uma operação de escrita não essencial por uma resposta simulada segura (*dry-run*), evitando alterações no ambiente externo (Singh e Madisetti, 2026);
- `HITL_PENDING`: Retém temporariamente a chamada em memória até que o operador humano aprove ou rejeite a operação (Subseção 3.2.3).

3.2 Módulo de Governança Determinística, RBAC e Máquina de Estados (PDP)

Normalizado o tráfego pelo Módulo 3.1, o segundo subproblema consiste em decidir de maneira determinística, com impacto de latência minímo e fora do modelo probabilístico, se uma chamada deve ser autorizada, bloqueada, rebaixada ou submetida a aprovação humana (Zhang et al., 2025; Singh e Madisetti, 2026). O Módulo de Governança Determinística, RBAC e Máquina de Estados de Sessão atua como o Ponto de Decisão de Política (PDP) do artefato, fechando a lacuna semântica (Bühler et al., 2026) e contendo a deriva de privilégios (Singh e Madisetti, 2026).

Este módulo recebe o Contexto de Interceptação $\mathcal{C}_{\text{req}}$, avalia seus atributos contra a matriz de políticas declarativas e o estado dinâmico da sessão, e emite o **Veredito de Governança** para o PEP, para o Módulo Forense (Seção 3.3) e para o Módulo de Observabilidade (Seção 3.4). Sua estrutura divide-se em três submódulos: Classificação Semântica e Matriz RBAC (Subseção 3.2.1), Máquina de Estados de Sessão L0–L3 (Subseção 3.2.2), e Elevação Supervisionada HITL (Subseção 3.2.3), ilustrados na Figura 3.3.

Figura 3.3: Diagrama de Fluxo de Decisão do Motor RBAC (PDP) e Máquina de Estados de Sessão (L0 - L3).



Fonte: Elaborado pelo autor (2026).

3.2.1 Classificação de Ferramentas e Matriz RBAC

Inspirado nos três perfis de acesso do framework *ACFIX* (Zhang et al., 2025) e no modelo declarativo do *AgentBound* (Bühler et al., 2026), este submódulo define as regras de permissão em um arquivo de políticas (`PolicyManifest`). Nesse manifesto, cada sessão do agente recebe um entre três papéis de acesso (RBAC):

- `Auditor` **(Usuário Restrito):** Limitado apenas a operações de consulta (`read`) sobre recursos públicos ou diretórios previamente autorizados;
- `Operator` **(Operador):** Autorizado a realizar consultas (`read`) e a executar ferramentas de escrita controlada (`write`), desde que os argumentos enviados respeitem as regras de validação;
- `Admin` **(Administrador):** Papel mais elevado, com permissão para acionar operações críticas ou destrutivas (`delete`), mas que exige aprovação humana em *runtime* antes da execução.

Para conectar esses papéis às ferramentas dos servidores MCP, o manifesto classifica cada ferramenta cadastrada conforme o impacto de sua operação: **leitura (`read`)**, quando apenas consulta dados sem alterar o estado do sistema; **escrita (`write`)**, quando

cria ou modifica arquivos e registros; ou **exclusão/crítica (`delete`)**, quando remove dados ou executa comandos sensíveis. Ao receber uma chamada, o gateway verifica primeiro se o papel da sessão tem permissão para o tipo de operação da ferramenta solicitada.

Entretanto, autorizar apenas o nome da ferramenta não é suficiente: mesmo que um agente com papel `Operator` tenha permissão para usar uma ferramenta de leitura de arquivos, ele ainda poderia ser induzido por uma injeção de prompt a ler chaves privadas do sistema. Por isso, após validar o papel, este submódulo inspeciona os argumentos (`params.arguments`) da chamada `tools/call` aplicando quatro regras práticas de validação:

1. **Restrição de Diretórios e Arquivos:** Verifica se os caminhos de arquivos informados pertencem exclusivamente às pastas permitidas, bloqueando tentativas de *Path Traversal* (`../`) e o acesso a arquivos sensíveis do *host* (como `~/.ssh/`, `.env` e `claude_desktop_config.json`) (Hasan et al., 2026; Bühler et al., 2026);

2. **Restrição de Domínios de Rede:** Permite requisições externas apenas para domínios listados na política do papel e bloqueia conexões para portas internas (`localhost`) (Jang e Lee, 2026; Hou et al., 2025);

3. **Bloqueio de Injeção de Comandos:** Inspeciona o texto dos argumentos por meio de expressões regulares e rejeita chamadas que contenham caracteres de encadeamento de comandos de terminal (`;`, `&&`, `|`, `‘`, `$()`) (Hasan et al., 2026);

4. **Limite de Taxa de Requisições (*Rate Limit*):** Limita a quantidade máxima de chamadas por sessão em um intervalo de tempo, interrompendo loops infinitos do agente ou tentativas de exfiltração de dados em massa.

3.2.2 Máquina de Estados de Sessão (Níveis L0 a L3)

Para impedir que sessões com permissão nominal de escrita sofram deriva de privilégios (*Privilege Drift*) após lerem conteúdos externos contaminados, o segundo submódulo mantém uma **Máquina de Estados Finitos de Segurança** para cada sessão s, operando nos quatro níveis progressivos propostos por Singh e Madisetti (Singh e Madisetti, 2026):

- **Nível L0 (*Baseline Aberto*):** Modo diagnóstico sem bloqueios, registrando eventos e telemetria para linha de base experimental;

- **Nível L1 (*Acesso Escopado*):** Restringe o catálogo em `tools/list` exclusivamente ao domínio funcional do papel atribuído à sessão;

- **Nível L2 (*Padrão Somente-Leitura Estrito*):** Mantém a sessão no estado `ScopedReadOnly`, permitindo consultas (`read`) válidas e convertendo qualquer chamada `write` ou `delete` em `DENY` ou `DOWNGRADE`;

- **Nível L3 (*HITL*):** Opera por padrão no estado `ScopedReadOnly` (idêntico ao L2), mas permite transições temporárias para mutações legítimas sob aprovação humana.

Formalmente, a máquina em nível L3 é definida pela quíntupla $\mathcal{M}_s = (Q, \Sigma, \delta, q_0, F)$ sobre os estados discretos `Init`, `ScopedReadOnly`, `PendingApproval`, `ElevatedWindow` e `Revoked`, com estado inicial $q_0 =$ `ScopedReadOnly`. Essa contenção induz diretamente a **Moldagem Comportamental** (*Behavioral Shaping*): ao receber recusas determinísticas quando tenta sair de `ScopedReadOnly` sem autorização, o modelo reajusta seu ciclo interno para concluir a tarefa com operações seguras (Singh e Madisetti, 2026).

3.2.3 Elevação Supervisionada com Janela Temporal (*HITL*)

Quando uma sessão em Nível L3 encontra-se em `ScopedReadOnly` e o agente emite uma chamada `tools/call` de escrita ou exclusão (`write` ou `delete`) que já foi aprovada pelas regras de papel e validação de argumentos da Subseção 3.2.1, este submódulo executa três passos (Singh e Madisetti, 2026; Hou et al., 2025):

1. **Suspensão em `PendingApproval`:** Emite o veredito `HITL_PENDING` ao PEP para reter o pacote JSON-RPC na fila de memória sem encerrar a conexão do cliente;

2. **Desafio Fora de Banda:** Envia o contexto completo da chamada (ferramenta, argumentos e nível de risco) a um canal de supervisão humana isolado do contexto do LLM;

3. **Janela Temporal (`ElevatedWindow`) e Reversão Automática:** Caso o supervisor rejeite a solicitação ou o tempo de espera expire, devolve `DENY` ao agente; caso aprove, transita a sessão para `ElevatedWindow` durante intervalo estrito pré definido ou execução única, libera a chamada e reverte imediatamente a sessão ao estado seguro `ScopedReadOnly`.

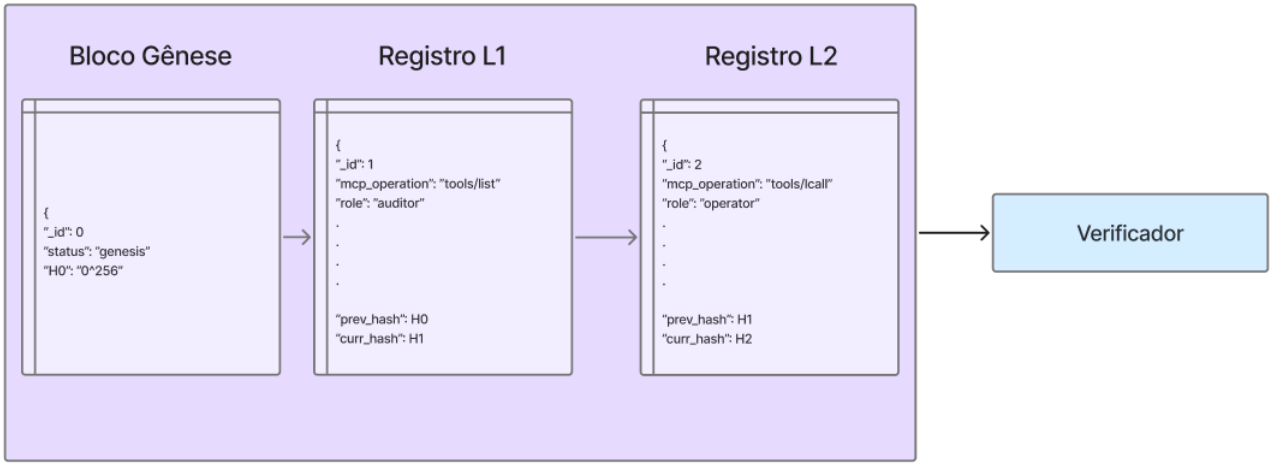
3.3 Módulo de Auditoria Forense Imutável e Verificação Criptográfica

O bloqueio preventivo de ações não autorizadas em *runtime* precisa ser acompanhado por mecanismos que garantam a perícia pós-incidente e a comprovação matemática da integridade do histórico operacional. Como os registros nativos gerados por servidores MCP locais são voláteis, despadronizados e vulneráveis à exclusão ou edição caso o ambiente seja comprometido, torna-se indispensável que o próprio gateway consolide uma trilha de auditoria externa à prova de adulteração, registrando tanto as chamadas permitidas quanto as tentativas de ataque contidas.

Para cumprir esse objetivo, o Módulo de Auditoria Forense Imutável e Verificação Criptográfica recebe o contexto normalizado da requisição proveniente do Módulo 3.1 e o

veredito emitido pelo Módulo 3.2, transformando-os em registros permanentes e encadeados. Seu funcionamento organiza-se em três submódulos complementares, ilustrados na Figura 3.4: o primeiro realiza a formatação padronizada do evento em estrutura JSONL com mascaramento de dados sensíveis (Subseção 3.3.1); o segundo calcula e acopla recursivamente o resumo criptográfico SHA-256 de cada linha ao elo anterior (Subseção 3.3.2); e o terceiro executa a validação linear da cadeia para detectar adulterações e reconstruir a sequência causal de incidentes (Subseção 3.3.3).

Figura 3.4: Arquitetura do Módulo de Auditoria Forense e Encadeamento Criptográfico SHA-256.



Fonte: Elaborado pelo autor (2026).

3.3.1 Serialização Canônica e Registro Estruturado (JSONL)

Para que o resumo criptográfico seja determinístico e reprodutível sem ambiguidades de formatação, o Submódulo de Serialização Canônica estrutura cada evento interceptado i como um documento JSON de linha única (*JSON Lines* - JSONL) com dez campos ordenados lexicograficamente:

1. `seq_id` (i): Contador inteiro estritamente monotônico iniciado em 1 na abertura da trilha;
2. `timestamp_utc` (T_i): Carimbo de tempo em padrão ISO 8601 UTC com resolução de microssegundos;
3. `session_trace_id` (ID_i): Tupla combinando o identificador da sessão MCP e o `trace_id` do OpenTelemetry;
4. `rbac_context` (R_i): Objeto contendo o papel ativo $r \in R$ e o nível de segurança (L0–L3);

5. `mcp_operation` (M_i): Método invocado (`tools/call`, `resources/read` ou `tools/list`), primitiva alvo e efeito (`read`, `write` ou `delete`);

6. `params_digest` (P_i): Argumentos normalizados da chamada com mascaramento de segredos via SHA-256 (Hasan et al., 2026; Hou et al., 2025);

7. `policy_verdict` (D_i): Decisão emitida pelo PDP (`ALLOW`, `DENY`, `DOWNGRADE` ou `HITL`);

8. `execution_metrics` (S_i): Código de status da resposta e latência interna do gateway em microssegundos (Δt_{gw});

9. `prev_hash` (H_{i-1}): Resumo hexadecimal SHA-256 de 64 caracteres do evento imediatamente anterior ($i-1$);

10. `curr_hash` (H_i): Resumo hexadecimal SHA-256 de 64 caracteres calculado sobre a concatenação canônica dos nove campos anteriores.

3.3.2 Encadeamento Criptográfico Recursivo (SHA-256)

Evoluindo a verificação estática de integridade por SHA-256 do *MCP Firewall* de Jang e Lee (Jang e Lee, 2026) para um fluxo contínuo em *runtime*, este submódulo mantém em memória protegida um acumulador monotônico que armazena o último resumo gerado (H_{last}) e o contador sequencial (i), inicializado com o bloco gênese $H_0 = 0^{256}$ (64 dígitos zeros hexadecimais). Para assegurar consistência estrita sob múltiplas sessões concorrentes no mesmo *host*, o cálculo de H_i (Equação 2.2) e a escrita de anexação no arquivo JSONL são executados dentro de um mecanismo de ordenação linear livre de condições de corrida, mantendo o tempo de cálculo em poucos microssegundos por evento.

3.3.3 Integridade e Reconstrução Causal

Executado sob demanda por auditores ou automaticamente na inicialização e manutenção do gateway (Hou et al., 2025; Jang e Lee, 2026), o algoritmo verificador percorre o arquivo JSONL do registro 1 ao n em tempo linear $O(n)$ validando três invariantes em cada linha k:

1. **Continuidade Sequencial:** Verifica se $k = k_{\text{anterior}} + 1$, detectando imediatamente a supressão de eventos ou truncamento da trilha;

2. **Integridade do Elo Anterior:** Confirma se o campo `prev_hash` do registro k coincide exatamente com o campo `curr_hash` do registro $k-1$ (ou com H_0 para $k = 1$);

3. **Autenticidade de Conteúdo:** Recalcula de forma independente o resumo SHA-256 sobre os campos canônicos de L_k e compara-o ao valor gravado em `curr_hash`, apontando o índice exato k caso qualquer caractere tenha sido adulterado.

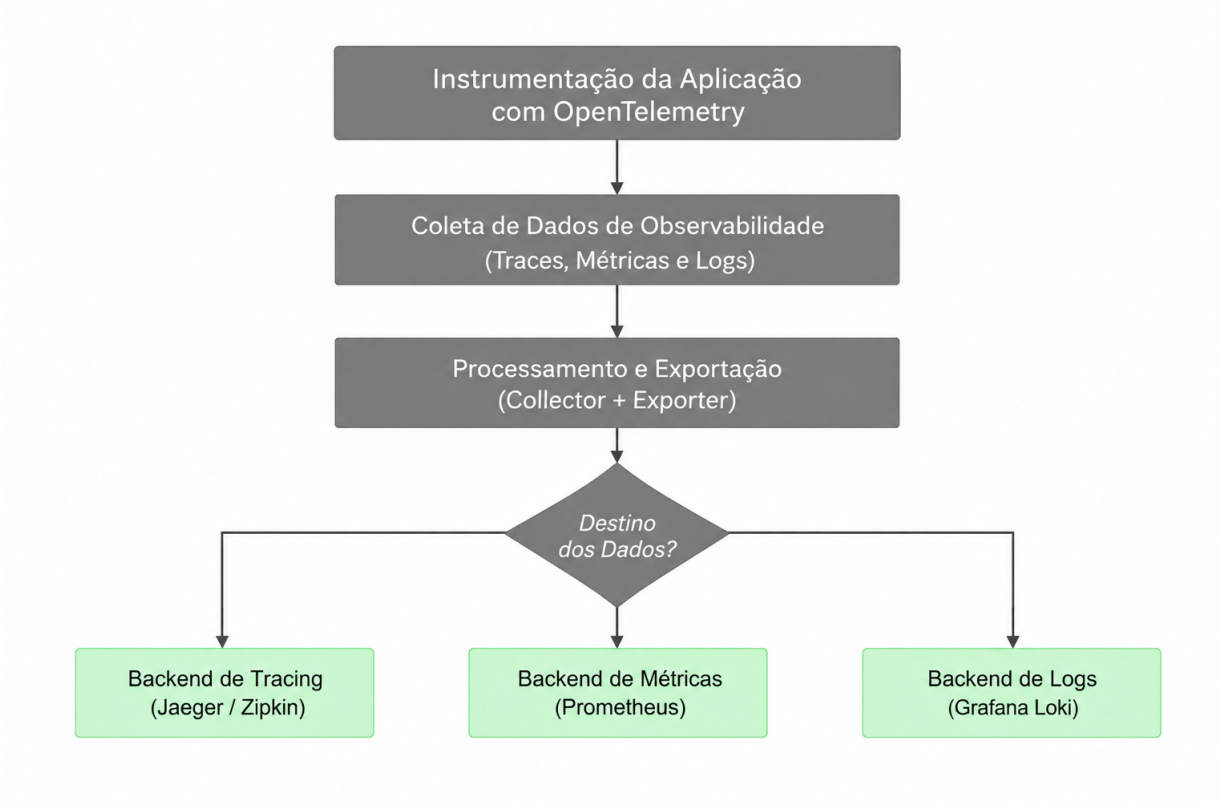
Quando íntegra, a trilha permite agrupar eventos por `session_trace_id` (ID_i) para reconstruir a linha do tempo exata de incidentes multi-etapas (Singh e Madisetti, 2026).

3.4 Módulo de Observabilidade Distribuída e Telemetria em *Runtime*

Enquanto o armazenamento forense em disco assegura a integridade histórica para auditorias posteriores, a operação contínua de agentes autônomos em produção exige visibilidade imediata sobre o comportamento das sessões, o volume de bloqueios de segurança e o custo de processamento adicionado pela mediação. Sem uma instrumentação padronizada em tempo real, administradores não conseguem identificar picos de ataques adversariais nem aferir se o gateway mantém sua latência dentro do limite sub-milissegundo estabelecido para o barramento MCP.

Para fornecer esse monitoramento contínuo sem impactar o caminho crítico das mensagens, o Módulo de Observabilidade Distribuída e Telemetria em *Runtime* instrumenta transversalmente todas as etapas internas do *Governance Gateway* segundo o padrão aberto **OpenTelemetry** (OTel), conforme esquematizado na Figura 3.5. Sua arquitetura divide-se em três submódulos: a propagação de contexto W3C no campo reservado `_meta` e a criação de segmentos internos de tempo para isolar a latência do gateway (Subseção 3.4.1); a coleta em memória de contadores de decisões RBAC, histogramas de desempenho e medidores de sessão (Subseção 3.4.2); e a correlação direta entre os identificadores de rastro e os resumos SHA-256 com exportação assíncrona via protocolo OTLP (Subseção 3.4.3).

Figura 3.5: Diagrama da Coleta de Métricas e Exportação OTLP



Fonte: Elaborado pelo autor (2026).

3.4.1 Propagação de Contexto W3C e Rastreamento Distribuído

Para correlacionar chamadas encadeadas através das fronteiras de processo sem poluir os argumentos funcionais das ferramentas MCP, este submódulo utiliza o objeto reservado `_meta` do JSON-RPC 2.0 para transportar os cabeçalhos da especificação *W3C Trace Context*:

- **Extração ou Geração de `traceparent`:** Ao interceptar uma requisição no Módulo 3.1, o submódulo inspeciona `params._meta.traceparent`. Caso o *host* já tenha enviado um contexto W3C (`00-traceId-parentId-flags`), o gateway adota o identificador de 128 bits recebido; caso contrário, instancia um novo identificador raiz na entrada do barramento;
- **Segmentos Internos (*Child Spans*):** Para isolar o custo computacional interno do gateway do tempo de resposta do servidor externo, são instanciados três *spans* filhos subordinados à requisição:
  1. `gateway.pdp.evaluate`: Cronometra a avaliação da matriz RBAC, das regras de validação e da máquina de estados;

2. `gateway.audit.commit`: Cronometra a serialização canônica JSONL e o cálculo recursivo SHA-256;
3. `mcp.server.forward`: Cronometra o tempo de processamento externo do servidor MCP de destino.

3.4.2 Métricas de Segurança e Desempenho

Para validar quantitativamente a hipótese de pesquisa (sobrecarga $< 1{,}0$ ms por chamada e contenção determinística de ataques (Singh e Madisetti, 2026)), este submódulo mantém três famílias de métricas OpenTelemetry em memória:

1. **Contadores:** Registram acumuladores monotônicos em `mcp.gateway.requests.total` (total de mensagens por método e papel RBAC), `mcp.gateway.policy.decisions` (total de vereditos `ALLOW`, `DENY`, `DOWNGRADE` e `HITL`) e `mcp.gateway.threats.blocked` (bloqueios por vetor de ataque);
2. **Histogramas de Latência:** Registram em `mcp.gateway.overhead.ms` a distribuição empírica do tempo exclusivo de mediação do gateway, permitindo extrair continuamente a mediana ($p50$) e os percentis de cauda ($p95$ e $p99$);
3. **Medidores de Estado:** Monitoram indicadores instantâneos em `mcp.gateway.active.sessions` (sessões ativas por nível L0–L3) e `mcp.gateway.hitl.pending` (chamadas aguardando aprovação humana em `PendingApproval`).

3.4.3 Correlação Forense-Telemetria e Exportação OTLP

Este submódulo acopla a observabilidade em tempo real à auditoria forense por meio de duas operações complementares:

- **Acoplamento Bidirecional (*Trace–Hash*):** Antes do cálculo de H_i no Módulo 3.3, injeta o `trace_id` e o `span_id` ativos no campo `session_trace_id` (ID_i) do registro JSONL e, no sentido inverso, anexa o resumo calculado H_i como atributo (`audit.sha256_hash`) ao *span* do OpenTelemetry;
- **Exportação Assíncrona OTLP:** Salva temporariamente os sinais em uma fila rápida que não trava o sistema e envia esses dados em blocos para ferramentas externas, como: Prometheus, Jaeger e Grafana.

3.5 Integração Conceitual do Modelo e Fundamentação nos Artigos Inspiradores

Apresentados individualmente os quatro módulos do *Governance Gateway*, esta seção consolida a visão integrada da solução conceitual, demonstrando o fluxo operacional

fim a fim de uma requisição (Subseção 3.5.1), o mapeamento detalhado entre os componentes do artefato e os artigos inspiradores revisados na literatura (Singh e Madisetti, 2026; Zhang et al., 2025; Bühler et al., 2026; Jang e Lee, 2026; Hou et al., 2025; Hasan et al., 2026; Guo et al., 2025) (Subseção 3.5.2), e a viabilidade de engenharia concorrente em máquinas virtuais (Subseção 3.5.3).

3.5.1 Fluxo Operacional Integrado Fim a Fim (Ciclo de Vida da Requisição)

A eficácia do modelo decorre da interação coordenada entre seus quatro módulos durante o ciclo de vida de uma sessão MCP.

O fluxo integrado desenvolve-se em seis passos encadeados:

1. **Estabelecimento de Sessão e Higienização de Catálogo:** O Módulo 3.1 intercepta `initialize`, instancia a sessão no estado `ScopedReadOnly` no Módulo 3.2 e filtra a resposta de `tools/list` contra ferramentas fora de escopo e *Tool Poisoning*;

2. **Interceptação de Chamada e Abertura de Rastro:** Captura `tools/call`, valida o JSON-RPC 2.0 e aciona o Módulo 3.4 para extrair ou criar o `traceparent` em `params._meta`;

3. **Avaliação Determinística de Política:** O PDP verifica o papel RBAC, os predicados de argumentos $c(args)$ e a compatibilidade do efeito (`read`, `write`, `delete`) com o nível da sessão (L0–L3);

4. **Contenção de Deriva de Privilégios ou Elevação HITL:** Emite `DENY` imediato em L2 ou retém a chamada em `PendingApproval` para confirmação humana em L3 caso uma mutação seja tentada fora da janela elevada;

5. **Persistência Forense Encadeada por SHA-256:** Grava atomicamente no Módulo 3.3 o registro JSONL L_i com resumo recursivo H_i;

6. **Imposição no Canal e Exportação OTLP:** Encaminha a chamada autorizada ou devolve a recusa e exporta assincronamente os *spans* e métricas coletados.

3.5.2 Mapeamento Conceitual com os Artigos Inspiradores

Cada decisão do *Governance Gateway* apoia-se diretamente nos métodos e arquiteturas validados pelos sete artigos revisados no Capítulo 2, que funcionam como alicerce para os módulos deste capítulo.

A Tabela 3.1 sintetiza a relação entre cada módulo do modelo conceitual, o artefato produzido, o subproblema atendido e os artigos inspiradores correspondentes.

Tabela 3.1: Mapeamento entre os módulos do Modelo Conceitual, artefatos, subproblemas e artigos inspiradores.

Módulo	**Artefato**	**Subproblema**	**Artigos Inspiradores**
3.1 Interceptação e Roteamento (PEP)	Contexto da requisição normalizado e catálogo de ferramentas filtrado.	Falta de mediação transparente no canal de comunicação.	Singh e Madisetti (Singh e Madisetti, 2026); Guo et al. (Guo et al., 2025); Hasan et al. (Hasan et al., 2026).
3.2 Governança, RBAC e Estados (PDP)	Veredito determinístico de autorização (`ALLOW`, `DENY`, `DOWNGRADE` ou `HITL`).	Mapeamento de ameaças, controle RBAC e moldagem comportamental.	Singh e Madisetti (Singh e Madisetti, 2026); Zhang et al. (Zhang et al., 2025); Bühler et al. (Bühler et al., 2026).
3.3 Auditoria Forense Imutável	Trilha de auditoria estruturada em JSONL encadeada via SHA-256.	Ausência de registros imutáveis contra adulteração.	Jang e Lee (Jang e Lee, 2026); Hou et al. (Hou et al., 2025); Hasan et al. (Hasan et al., 2026).
3.4 Observabilidade em *Runtime*	Rastreamento distribuído (*spans*) e métricas exportadas via OpenTelemetry.	Falta de telemetria em tempo real e aferição de latência < 1 ms.	Jang e Lee (Jang e Lee, 2026); Singh e Madisetti (Singh e Madisetti, 2026); Hou et al. (Hou et al., 2025).

Fonte: Elaborado pelo autor (2026).

De forma complementar à síntese apresentada na tabela, detalha-se a seguir como os conceitos e métodos de cada artigo de base foram incorporados na concepção do *Governance Gateway*:

- **Singh e Madisetti (Singh e Madisetti, 2026):** Fornece a base para a mediação em *runtime* no barramento JSON-RPC, incluindo a interceptação transparente de chamadas, a máquina de estados em quatro níveis (L0 a L3), a separação entre operações de leitura e escrita, a elevação temporária supervisionada (HITL) e o conceito de moldagem comportamental;
- **Zhang et al. (Zhang et al., 2025):** Fundamenta a retirada das decisões de acesso do modelo probabilístico por meio de uma matriz determinística de RBAC, inspirando a divisão nos três papéis do gateway (`Admin`, `Operator` e `Auditor`) e a validação conjunta entre papel, operação e argumentos;
- **Bühler et al. (Bühler et al., 2026):** Embasa o conceito da lacuna semântica para justificar a inspeção no nível do protocolo JSON-RPC e inspira, a partir do

`AgentManifest`, a criação do manifesto declarativo de políticas (`PolicyManifest`) com restrições de diretórios e domínios de rede;

- **Jang e Lee (Jang e Lee, 2026):** Inspira o uso da função criptográfica SHA-256 para verificação de integridade (evoluída neste modelo para um encadeamento contínuo de registros JSONL) e fundamenta os requisitos de monitoramento em tempo real com OpenTelemetry;
- **Hou et al. (Hou et al., 2025):** A análise das vulnerabilidades de ciclo de vida e da ausência de controles na fase de manutenção orienta o encapsulamento local de processos *stdio*, a expiração automática da janela de privilégio elevado e a rotina de verificação periódica dos logs;
- **Hasan et al. (Hasan et al., 2026) e Guo et al. (Guo et al., 2025):** Os levantamentos empíricos sobre envenenamento de descrições, exposição de segredos, injeção de comandos e predominância dos canais *stdio* e *Streamable HTTP* fundamentam o adaptador de transporte dual, a higienização de catálogos e o mascaramento de credenciais na trilha forense.

3.5.3 Viabilidade de Engenharia Concorrente em Máquinas Virtuais

Para que o modelo conceitual cumpra a meta de adicionar menos de 1,0 ms de latência por chamada JSON-RPC sob múltiplas sessões simultâneas, sua arquitetura interna foi projetada segundo o paradigma de **concorrência orientada a passagem de mensagens e isolamento de estado**. Em máquinas virtuais de alta concorrência (como a máquina virtual BEAM do ecossistema Elixir/Erlang ou a JVM com *Virtual Threads* em Java), cada componente mapeia-se para primitivas leves de execução:

1. **Isolamento de Sessão por Processo Leve ou Ator:** Cada sessão ativa $s \in S$ mantém em memória privada seu estado atual $q \in Q$, seu papel RBAC e o temporizador da janela HITL (Δt), eliminando travas globais (*locks*) de sincronização durante a avaliação do PDP;
2. **Tabelas de Políticas Imutáveis em Memória:** Os manifestos declarativos e matrizes RBAC são compilados na inicialização em tabelas de busca $O(1)$ compartilhadas em modo somente-leitura, permitindo a avaliação em dezenas de microssegundos;
3. **Serialização Atômica Não-Bloqueante:** O acumulador recursivo SHA-256 opera como um sequenciador dedicado em memória e o exportador OTLP funciona de modo assíncrono via buffer circular, fora do caminho crítico de resposta ao agente.

Bibliografia

BÜHLER, C. et al. AgentBound: Securing execution boundaries of AI agents. **Proceedings of the ACM on Software Engineering**, ACM New York, NY, USA, v. 3, n. FSE, p. 1–24, 2026.

GUO, H. et al. A measurement study of model context protocol ecosystem. **arXiv preprint arXiv:2509.25292**, 2025.

HASAN, M. M. et al. Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. **ACM Transactions on Software Engineering and Methodology**, ACM New York, NY, USA, v. 35, n. 2, p. 1–51, 2026.

HOU, X. et al. Model context protocol (MCP): Landscape, security threats, and future research directions. **arXiv preprint arXiv:2503.23278**, 2025.

JANG, M.; LEE, K. Hierarchical approaches for development of an MCP security framework. **IEEE Access**, IEEE, v. 14, p. 77148–77164, 2026.

SINGH, G.; MADISETTI, V. K. MCP-Secure: A runtime access control layer for privilege-aware LLM agent tooling. **IEEE Open Journal of the Computer Society**, IEEE, v. 7, p. 504–514, 2026.

ZHANG, L. et al. ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts. **IEEE Transactions on Software Engineering**, IEEE, v. 51, n. 9, p. 2512–2532, 2025.